B.M.S. COLLEGE OF ENGINEERING

(Autonomous college under VTU)

Bull Temple Rd, Basavanagudi, Bengaluru, Karnataka 560019 2023-2025

Department of Computer Applications

Report is submitted for fulfillment of AAT work in the subject

"Big Data Analytics"
(22MCA2PEBD)

By

Tejaswini N
(1BM23MC103)

Under the Guidance

Prof. S Shilpa
(Assistant Professor)

CONTENTS

SL. No.	Programs	Page No.
1	Load the u.data into the new RDD.	2
2	Change the RDD to a Dataframe.	3
3	Return the schema of this DataFrame.	3
4	Register the DataFrame as a temp u_data table.	4
5	Display the contents of newly created u_data table.	4
6	Show the numbers of items reviewed by each user in the newly created u_data table.	5
7	Show the numbers of users reviewed each item in the newly created u_data table.	6
8	Load the u.user into a new RDD.	7
9	Change the RDD to a Dataframe.	7-8
10	Register the DataFrame as a temp u_user table.	9
11	Display the contents of newly created user table.	9
12	Count the number of user in the u_user table gender wise.	10
13	Join u_data table and u_user tables based on user id.	11-12

You will load data into Spark DataFrames, and explore tabular data with Spark SQL. This concept can be reused as the foundation to solve any type of business problem.

MovieLens datasets were collected by the GroupLens Research Project at the University of minnesota. It represents users' reviews of movies.

This data set consists of:

* 100,000 ratings (1-5) from 943 users on 1682 movies.

* Each user has rated at least 20 movies.

* Simple demographic info for the users (age, gender, occupation, zip)

u.data -- The full u data set, 100000 ratings by 943 users on 1682 items.

Each user has rated at least 20 movies. Users and items are numbered consecutively from 1. The data is randomly ordered. This is a tab separated list of

user id item id rating timestamp.

The time stamps are unix seconds since 1/1/1970 UTC

u.user -- Demographic information about the users; this is a pipe separated list of

user id | age | gender | occupation | zip code

The user ids are the ones used in the u.data data set.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("MovieLens Data Analysis").getOrCreate()
sc = spark.sparkContext
```

```
>>> from pyspark.sql import SparkSession
>>> spark = SparkSession.builder.appName("MovieLens Data Analysis").getOrCreate()
>>>
>>> sc = spark.sparkContext
```

1.load the u.data into the new RDD.

#Load u.data into an RDD

>> ratings_rdd = sc.textFile("/home/mca/Downloads/u.data")

Display the first few records

>>> print(ratings_rdd.take(5))

```
>>> ratings_rdd = sc.textFile("/home/mca/Downloads/u.data")
>>>
>>> print(ratings_rdd.take(5))
['196\t242\t3\t881250949', '186\t302\t3\t891717742', '22\t377\t1\t878887116', '244\t51\t2\t880606923', '166\t346\t1\t886397596']
```

Step 4: Parse the RDD into a structured format

parsed_ratings_rdd = ratings_rdd.map(lambda line:line.split('\t'))

>>> print(parsed_ratings_rdd.take(5))

```
>>> print(parsed_ratings_rdd.take(5))
[['196', '242', '3', '881250949'], ['186', '302', '3', '891717742'],
```

```
['22', '377', '1', '878887116'], ['244', '51', '2', '880606923'], ['166', '346', '1', '886397596']]
```

2.Change the RDD to a Dataframe.

Convert RDD to DataFrame with appropriate column names

>>> df_ratings = parsed_ratings_rdd.toDF(["user_id", "item_id", "rating", "timestamp"])

OR

```
# Define the schema
rating_schema = StructType([
    StructField("userId", IntegerType(), True),
    StructField("movieId", IntegerType(), True),
    StructField("rating", IntegerType(), True),
    StructField("timestamp", LongType(), True)
])

# Convert the parsed RDD to a DataFrame
ratings_df = spark.createDataFrame(parsed_ratings_rdd, schema=rating_schema)
```

3.Return the schema of this DataFrame.

```
# Display the schema of the DataFrame
>>> df_ratings.printSchema()
```

```
>>> df_ratings.printSchema()
root
 |-- user_id: string (nullable = true)
 |-- item_id: string (nullable = true)
 |-- rating: string (nullable = true)
 |-- timestamp: string (nullable = true)
```

4.Register the DataFrame as a temp u_data table.

```
>>> df_ratings.createOrReplaceTempView("u_data")
```

5 Display the contents of newly created u_data table

```
>>> df_ratings.show()
```

OR

```
>>> spark.sql("SELECT * FROM u_data").show()
```

```
>>> df_ratings.show()
+-------+-------+------+---------+
|user_id|item_id|rating|timestamp|
+-------+-------+------+---------+
|    196|    242|     3|881250949|
|    186|    302|     3|891717742|
|     22|    377|     1|878887116|
|    244|     51|     2|880606923|
|    166|    346|     1|886397596|
|    298|    474|     4|884182806|
|    115|    265|     2|881171488|
|    253|    465|     5|891628467|
|    305|    451|     3|886324817|
|      6|     86|     3|883603013|
|     62|    257|     2|879372434|
|    286|   1014|     5|879781125|
|    200|    222|     5|876042340|
|    210|     40|     3|891035994|
|    224|     29|     3|888104457|
|    303|    785|     3|879485318|
|    122|    387|     5|879270459|
|    194|    274|     2|879539794|
|    291|   1042|     4|874834944|
|    234|   1184|     2|892079237|
+-------+-------+------+---------+
only showing top 20 rows
```

6.Show the numbers of items reviewed by each user in the newly created u_data table.

```
>>> review_counts_by_user = spark.sql("""
...     SELECT user_id, COUNT(item_id) AS num_items_reviewed
...     FROM u_data
...     GROUP BY user_id
...     ORDER BY num_items_reviewed DESC  """)

>>> review_counts_by_user.show()
```

OR

Show the number of items reviewed by each user

```
>>> spark.sql("SELECT user_id, COUNT(*) AS num_reviews FROM u_data GROUP BY user_id").show()
```

```
>>> review_counts_by_user.show()
```

```
>>> review_counts_by_user.show()
+-------+------------------+
|user_id|num_items_reviewed|
+-------+------------------+
|    405|               737|
|    655|               685|
|     13|               636|
|    450|               540|
|    276|               518|
|    416|               493|
|    537|               490|
|    303|               484|
|    234|               480|
|    393|               448|
|    181|               435|
|    279|               434|
|    429|               414|
|    846|               405|
|      7|               403|
|     94|               400|
|    682|               399|
|    308|               397|
|     92|               388|
|    293|               388|
+-------+------------------+
only showing top 20 rows
```

7.Show the numbers of users reviewed each item in the newly created u_data table

```
>>> review_counts_by_item = spark.sql("""
...     SELECT item_id, COUNT(user_id) AS num_users_reviewed
...     FROM u_data
...     GROUP BY item_id
...     ORDER BY num_users_reviewed DESC  """)
```

```
>>> review_counts_by_item.show()
```

OR

Show the number of users who reviewed each item

>>> spark.sql("SELECT item_id, COUNT(*) AS num_reviews FROM u_data GROUP BY item_id").show()

```
>>> review_counts_by_item.show()
+-------+------------------+
|item_id|num_users_reviewed|
+-------+------------------+
|     50|               583|
|    258|               509|
|    100|               508|
|    181|               507|
|    294|               485|
|    286|               481|
|    288|               478|
|      1|               452|
|    300|               431|
|    121|               429|
|    174|               420|
|    127|               413|
|     56|               394|
|      7|               392|
|     98|               390|
|    237|               384|
|    117|               378|
|    172|               367|
|    222|               365|
|    313|               350|
+-------+------------------+
only showing top 20 rows
```

8.Load the u.user into a new RDD.

Load u.user into an RDD

>>> users_rdd = sc.textFile("/home/mca/Downloads/u.user")

>>> print(users_rdd.take(5))

```
>>> print(users_rdd.take(5))
['1|24|M|technician|85711', '2|53|F|other|94043',
>>>
```

```
'3|23|M|writer|32067', '4|24|M|technician|43537', '5|33|F|other|15213']
```

9.Change the RDD to a Dataframe.

```
# Parse the RDD into a structured format
>>> parsed_users_rdd = users_rdd.map(lambda line: line.split('|'))

 # Convert RDD to DataFrame with appropriate column names
>>> df_users = parsed_users_rdd.toDF(["user_id", "age", "gender", "occupation", "zip_code"])

>>> df_users.printSchema()
```

```
>>> df_users.printSchema()
root
 |-- user_id: string (nullable = true)
 |-- age: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- occupation: string (nullable = true)
 |-- zip_code: string (nullable = true)
```

OR

```
# Define the schema

>>> from pyspark.sql.types import StructType, StructField, IntegerType, StringType, LongType
>>> user_schema = StructType([
...         StructField("userId", IntegerType(), True),
...         StructField("age", IntegerType(), True),
...         StructField("gender", StringType(), True),
...         StructField("occupation", StringType(), True),
...         StructField("zip", StringType(), True)

>>> # Convert the parsed RDD to a DataFrame
>>> users_df = spark.createDataFrame(parsed_users_rdd, schema=user_schema)
```

>>> users_df.printSchema()

```
>>> df_users.printSchema()
root
 |-- user_id: string (nullable = true)
 |-- age: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- occupation: string (nullable = true)
 |-- zip_code: string (nullable = true)
```

10. Register the DataFrame as a temp u_user table.

Register the DataFrame as a temporary view

>>> df_users.createOrReplaceTempView("u_user")

11.Display the contents of newly created user table

>>> # Display the contents of the u_user table

>>> df_users.show()

OR

>>> spark.sql("SELECT * FROM df_users").show()

```
>>> df_users.show()
+-------+---+------+-------------+--------+
|user_id|age|gender|   occupation|zip_code|
+-------+---+------+-------------+--------+
|      1| 24|     M|   technician|   85711|
|      2| 53|     F|        other|   94043|
|      3| 23|     M|       writer|   32067|
|      4| 24|     M|   technician|   43537|
|      5| 33|     F|        other|   15213|
|      6| 42|     M|    executive|   98101|
|      7| 57|     M|administrator|   91344|
|      8| 36|     M|administrator|   05201|
|      9| 29|     M|      student|   01002|
|     10| 53|     M|       lawyer|   90703|
|     11| 39|     F|        other|   30329|
|     12| 28|     F|        other|   06405|
|     13| 47|     M|     educator|   29206|
|     14| 45|     M|    scientist|   55106|
|     15| 49|     F|     educator|   97301|
|     16| 21|     M|entertainment|   10309|
|     17| 30|     M|   programmer|   06355|
|     18| 35|     F|        other|   37212|
|     19| 40|     M|    librarian|   02138|
|     20| 42|     F|    homemaker|   95660|
+-------+---+------+-------------+--------+
only showing top 20 rows
```

12.Count the number of user in the u_user table gender wise

```
>>> gender_counts = spark.sql("""
...     SELECT gender, COUNT(user_id) AS num_users
...     FROM u_user
...     GROUP BY gender
... """)

>>> gender_counts.show()
```

```
>>> gender_counts.show()
+------+---------+
|gender|num_users|
+------+---------+
|     F|      273|
|     M|      670|
+------+---------+
```

OR

```
>>> # Count the number of users by gender
>>> spark.sql("SELECT gender, COUNT(*) AS num_users FROM u_user GROUP BY gender").show()
```

```
>>> gender_counts.show()
+------+---------+
|gender|num_users|
+------+---------+
|     F|      273|
|     M|      670|
+------+---------+
```

13. Join u_data table and u_user tables based on userid

```
# Register DataFrames as temporary views

>>> df_ratings.createOrReplaceTempView("u_data")
>>> df_users.createOrReplaceTempView("u_user")

>>> joined_df = spark.sql("""
...     SELECT d.user_id, d.item_id, d.rating, d.timestamp, u.age, u.gender, u.occupation, u.zip_code
...     FROM u_data d
...     JOIN u_user u
...     ON d.user_id = u.user_id
... """)

>>> joined_df.show()
```

OR

```
>>> joined_df = df_ratings.join(df_users, on="user_id", how="inner")
>>> joined_df.show()
```

OR

```
>>> joined_df = spark.sql("""
...     SELECT u.user_id, u.item_id, u.rating, u.timestamp,
...     us.age, us.gender, us.occupation, us.zip_code
...     FROM u_data u
...     JOIN u_user us
...     ON u.user_id = us.user_id
... """)

>>> joined_df.show()
```

```
>>> joined_df.show()
+-------+-------+------+---------+---+------+-------------+--------+
|user_id|item_id|rating|timestamp|age|gender|   occupation|zip_code|
+-------+-------+------+---------+---+------+-------------+--------+
|    296|    705|     5|884197193| 43|     F|administrator|   16803|
|    296|    508|     5|884196584| 43|     F|administrator|   16803|
|    296|     20|     5|884196921| 43|     F|administrator|   16803|
|    296|    228|     4|884197264| 43|     F|administrator|   16803|
|    296|    222|     5|884196640| 43|     F|administrator|   16803|
|    296|    429|     5|884197330| 43|     F|administrator|   16803|
|    296|    855|     5|884197352| 43|     F|administrator|   16803|
|    296|    248|     5|884196765| 43|     F|administrator|   16803|
|    296|    258|     5|884196469| 43|     F|administrator|   16803|
|    296|    242|     4|884196057| 43|     F|administrator|   16803|
|    296|     48|     5|884197091| 43|     F|administrator|   16803|
|    296|    286|     5|884196209| 43|     F|administrator|   16803|
|    296|    272|     5|884198772| 43|     F|administrator|   16803|
|    296|    510|     5|884197264| 43|     F|administrator|   16803|
|    296|    275|     4|884196555| 43|     F|administrator|   16803|
|    296|    427|     5|884198772| 43|     F|administrator|   16803|
|    296|     83|     5|884199624| 43|     F|administrator|   16803|
|    296|    961|     5|884197287| 43|     F|administrator|   16803|
|    296|    544|     4|884196938| 43|     F|administrator|   16803|
|    296|     32|     4|884197131| 43|     F|administrator|   16803|
+-------+-------+------+---------+---+------+-------------+--------+
only showing top 20 rows
```